

ADR-06 — Implementación de paginación en consultas del sistema

Estado

Propuesto

Contexto

El sistema **ERP Iglesias** gestiona diferentes entidades como:

- personas
- cursos
- inscripciones
- ofrendas
- pagos

A medida que el sistema sea utilizado durante más tiempo, el volumen de información almacenada en la base de datos aumentará considerablemente.

Si las consultas retornan **todas las filas de una tabla**, se pueden presentar problemas como:

- alto consumo de memoria en el backend
- tiempos de respuesta más altos
- sobrecarga en la red al transferir grandes cantidades de datos
- interfaces de usuario difíciles de navegar

En aplicaciones modernas es una práctica común implementar **paginación** para limitar la cantidad de registros retornados por consulta.

Problema

Si los endpoints REST retornan listas completas de registros:

- el backend debe procesar grandes volúmenes de datos
- el frontend recibe más información de la necesaria
- la experiencia de usuario se deteriora cuando las listas son muy largas
- el rendimiento del sistema disminuye conforme crece la base de datos

Esto afecta tanto **el rendimiento del sistema** como **la usabilidad de la interfaz**.

Decisión Arquitectónica

Se propone implementar **paginación en las consultas del sistema**, tanto en el **backend** como en el **frontend**.

Los endpoints REST deberán permitir solicitar los datos en **bloques de registros (páginas)** en lugar de retornar la lista completa.

Arquitectura propuesta:



Patrón Aplicado

Pagination Pattern

Este patrón permite dividir grandes conjuntos de datos en **subconjuntos manejables (páginas)** para mejorar el rendimiento y la experiencia de usuario.

Principio SOLID Aplicado

Single Responsibility Principle (SRP)

Cada capa del sistema tendrá una responsabilidad específica:

- Backend → gestionar paginación y consultas eficientes
- Frontend → controlar navegación entre páginas
- Base de datos → retornar solo los registros necesarios

Cambio Concreto Propuesto

Los endpoints del sistema deberán aceptar parámetros de paginación.

Ejemplo de endpoint:

```
GET /api/people?page=1&size=10
```

Parámetros:

page → número de página solicitada
size → cantidad de registros por página

Ejemplo de Consulta Paginada

Consulta al backend:

```
GET /api/people?page=1&size=10
```

Respuesta:

```
{
  "page": 1,
  "size": 10,
  "totalElements": 45,
  "totalPages": 5,
  "content": [ ... registros ... ]
}
```

Esto permite al frontend mostrar únicamente los registros correspondientes a la página solicitada.

Ejemplo para Entenderlo

La paginación funciona dividiendo los datos en bloques.

```
Página 1 → registros 1 a 10
Página 2 → registros 11 a 20
Página 3 → registros 21 a 30
```

Este comportamiento es similar al de plataformas como **Netflix**, donde inicialmente se muestran algunos elementos y luego se pueden cargar más resultados.

Beneficios Arquitectónicos

La implementación de paginación proporciona:

- mejor rendimiento del sistema
- menor consumo de recursos en el backend

- menor volumen de datos transferidos
- mejor experiencia de usuario en interfaces con listas grandes
- mayor escalabilidad del sistema a futuro

Impacto en el Sistema

La implementación de paginación no altera la funcionalidad del sistema, pero modifica la forma en que los datos son consultados y presentados.

Los clientes deberán solicitar los datos utilizando los parámetros de paginación definidos en la API.

Resultado Arquitectónico

Arquitectura final propuesta:

```
Frontend
↓
API REST (paginación)
↓
Consultas paginadas
↓
Base de datos
```

Esta decisión mejora la eficiencia del sistema al manejar grandes volúmenes de datos de forma más controlada y escalable.